

コードの時系列変化を考慮した 機械学習に基づく欠陥混入の低減手法

鈴木研究室 2410064

笹川 尋翔

1.1 背景: ソフトウェア開発と欠陥リスク

- 開発サイクルの高速化: CI/CDの普及によりリリース後もソースコードの継続的かつ頻繁な変更が不可欠となった
- 修正コストの増大: 欠陥の発見が遅れるほど、修正に必要な工数とコストは指数関数的に増大する
- リファクタリングの懸念: 76%の開発者が、変更作業による意図しないバグ混入のリスクを認識している^[1]

[1] M. Kim et al., "An empirical study of refactoring challenges and benefits at microsoft," 2014'

1.2 研究の目的: 意思決定支援モデルの構築

- 早期検出とリソース配分: 特徴量の時間的变化を通じてバグ混入リスクを特定し、限られたレビュー労力を効率的に配分する意思決定支援モデルを提示する
 1. 構成要素の統合: メソッド単位の局所的変化とコミット単位の全体的変化を組み合わせる
 2. 不規則な時系列の考慮: コミットを点過程データとして扱い、直前状態からの差分を学習する
 3. レビュー労力の最適化: 規模と複雑さに着目した労力指標を定義し、発見率を向上させる

1.3 関連研究と既存手法の課題

- 構造的メトリクスの限界: 循環的複雑度^[2]やコード行数などは静的な状態を示すが、開発プロセスの動的な特徴を反映できない
- 変更メトリクスの発展: 変更の規模や頻度が強い予測因子となることが示されているが^[3]、時系列構造を明示的に考慮した手法は少ない
- レビュー効率化: 20%の労力で35%の欠陥特定が可能という報告があるが^[4]、労力の推定精度に課題が残る

[2] T. J. McCabe, "A complexity measure," 1976

[3] T. L. Graves et al., "Predicting fault incidence using software change history," 2000

[4] Y. Kamei et al., "A large-scale empirical study of just-in-time quality assurance," 2013'

2.1 提案手法1: 不規則な時系列情報の活用

- 点過程データとしての解析: コミットを不規則に発生するイベントと定義し、発生タイミングが持つリスク情報を活用する
- 差分ベースの特徴量抽出: 絶対値ではなく、直前の状態からの変化量に着目する
 - 例: 現在の行数ではなく何行増えたか、構造がどれほど複雑化したかを重視
- 予兆の検出: 短期間での急激な複雑化など、静的な解析では捉えきれない欠陥の予兆を識別可能にする

2.2 提案手法2: マルチレベル変更メトリクス

- 局所的視点（メソッド単位）：コード行数、トークン数、循環的複雑度の各変化量を用いる
- 全体的視点（コミット単位）：変更ファイル数、追加・削除行数、およびエントロピーを用いた変更の分散度を用いる
- 相補的効果: 単独の要素では見落とされがちな局所的な修正がシステム全体に及ぼすリスクを統合的に評価する

2.3 特徴量設計: メソッド識別子の高度な処理

- 機能情報の抽出: parse や validate 等の識別子は欠陥発生率と関連が深い
ため、これを特徴量化する
- トークン分割: キャメルケース等に基づき分解し、単語レベルでの類似性を認識可能にする
 - 例: getUsername は user と name に分解。
- TF-IDFによるベクトル化: 頻出語を抑制しつつ、特定の機能を示す重要なキーワードに重みを付与し、汎化性能を高める

2.4 レビュー優先度付け: 労力制約下の最適化

- 補正済みレビュー労力: 変更行数と広がりを経統合し、対数変換により巨大な変更の影響を緩和した指標を定義する
 - $W_i = \log_2(C_i \times N_i^{\overline{H}_i} + 1)$
- 組み合わせ最適化: レビュー対象の選定をナップサック問題として捉え、労力の上限内で期待値を最大化する
- 貪欲法による求解: 単位労力あたりの欠陥混入確率が高い密度順に選定する

3.1 実験環境と対象プロジェクト

- データセット: 欠陥の混入から修正までの対応関係を含む BugHunter データセット^[5]を採用
- 対象プロジェクト: Elasticsearch, Hazelcast, Netty, OrientDB, Neo4j
- 機械学習アルゴリズム: 非線形関係の抽出と解釈性に優れたランダムフォレストを使用
- 評価手法: 10分割交差検証、F1スコア、AUC、および McNemar 検定等による有意差検証を行う

[5] R. Ferenc et al., "An automatically created novel bug dataset and its validation in bug prediction," 2020'

3.2 実験結果1: 予測性能の向上

- 精度の改善幅: 全プロジェクトで性能が一貫して向上し、F1スコアの平均改善幅は0.21を達成した
- 高い識別性能: 最終的なAUCは全プロジェクトで0.91を超え、高い識別能力を確認した
- 主要因子: 特徴量重要度の分析により、コミット単位の追加行数や変更ファイル数が特に予測に寄与することが判明した

3.3 実験結果2: レビュー効率の劇的な改善

- 労力削減効果: 提案手法により、全欠陥の70から75%をわずか20%のレビュー労力で特定することに成功した
- 既存手法との比較: 労力40%時点での欠陥発見率は平均87.0%に達し、既存手法（75.8%）から11.3ポイントの改善を達成した
- 統計的有意性: 40%時点において、既存手法に対し統計的に有意な改善を確認した

4.1 考察1: プロジェクト特性と不均衡データ

- 相補的關係: 構造的メトリクスのみでは予測が困難なプロジェクトほど、時系列情報の追加による改善幅が大きくなる
- 不均衡データへの有効性: 陽性クラスの割合が低いほど改善効果が顕著であり、変化の特徴が少数派の識別に有効に機能している
- 汎用性の実証: 多様なドメイン（検索、DB、通信等）において一貫して高い性能を発揮した

4.2 考察2: 変更規模と欠陥確率の逆転現象

- 分析結果: PDP分析により、変更規模が小さいほど欠陥混入確率が高いという傾向が観察された
- 要因の考察:
 1. 小規模な変更は外科手術的なバグ修正であることが多く、新たな不具合を誘発しやすい^[6]
 2. 小規模な変更はレビューが容易なため、潜在的な欠陥が発見されやすく記録に残る可能性が高い

[6] Abram Hindle et al., "What do large commits tell us? a taxonomical study of large commits," 2008

4.3 評価と手法の制約

- 正解ラベルの信頼性: 欠陥の特定に使用したSZZアルゴリズムには、特定の修正を見落とすなどの課題が存在する^[7]
- 言語・ドメイン依存性: 本研究は Java プロジェクトを対象としており、異なる言語特性への適用には再検証が必要である
- 解釈性の優先: 精度の最大化だけでなく、開発者がなぜ予測されたかを理解できる透明性を重視して構築した

[7] S. Herbold et al., "Problems with szz and features," 2022

5.1 結論

- 時系列データの有効性: コミット間の不規則な変化を捉えることで、従来の静的な解析を大幅に上回る精度での予測を可能にした
- 実務への貢献: 労力制約を考慮した優先度付けにより、現実の開発環境において品質保証を効率化するモデルを提示した
- 目的の達成: 多層メトリクスの統合、時系列考慮、労力最適化のすべてを達成した

5.2 今後の展望: 変更目的の推定と予防

- 原因の特定: 変更規模だけでなく、機能追加やリファクタリングといった変更の目的と欠陥原因を関連付ける必要がある
- 推定要素の体系化: コミットメッセージ、コード構造、変更文脈、開発者履歴などの情報を統合しモデルを構築する
- 品質の予防へ: 欠陥が混入した原因を突き止めることで、具体的な予防ガイドラインの提示を目指す